ITF MANUAL TESTING COURSE

Human Resources Database

Managing Database Structure and Writing SQL Queries

Project overview

Item	Details
Student	Valentina Hatos
Role	QA Tester
Application	MySQL database “humanresources”
Module tested	Database structure, data changes and SQL queries
Tools	MySQL Workbench
Date	September 2022

Contents

1. Introduction....3

2. Creating the Databases and the Tables....3

3. Changing the Table Structure....3

4. Inserting Data....4

5. Queries....4

5.1 Selecting and sorting....4

5.2 Filters and functions....4

5.3 Changing the data....5

6. Linking the Tables....5

7. Join Queries....6

8. Conclusion....6

1. Introduction

This project is an SQL exercise from the ITF Manual Testing course. I created a database for a human resources department. The database stores employees and the rewards they receive.

In this exercise, I focused on the structure of the database. I created and deleted databases and tables, added and removed columns, and changed keys. Then I inserted data, changed it and wrote queries, including joins between two tables. The exercise has 22 tasks.

2. Creating the Databases and the Tables

Tasks 1 and 2: I created two databases, humanresources and marketing. Then I deleted the marketing database with DROP DATABASE.

```
create database humanresources;
create database marketing;

drop database marketing;
```

Task 3: I created three tables in the humanresources database.

Table	What it stores
employee	The id, the name, the salary and the joining date of each employee
reward	The rewards of the employees: the date and the amount
holidays	The name and the date of each holiday

The Employee_id column is the primary key and increases automatically.

```
create table employee (
Employee_id int auto_increment primary key,
First_name varchar(50),
Last_name varchar(50),
Salary int,
Joining_date date
);

create table reward (
Employee_ref_id int,
date_reward date,
amount int
);

create table holidays (
holiday_name varchar(50),
holiday_date date
);
```

3. Changing the Table Structure

Task 4: I added two new columns to the employee table: Birthdate and Departament.

```
alter table employee add Birthdate date;
alter table employee add Departament varchar(50);
```

Task 5: I removed the Birthdate column.

```
alter table employee drop Birthdate;
```

Task 6: I deleted the holidays table.

```
drop table holidays;
```

After these changes, the database has two tables: employee and reward.

4. Inserting Data

Task 7: I inserted 6 employees from four departments: IT, Banking, Insurance and Finance. Then I updated the salary of the first employee. I also inserted 4 rewards.

```
insert into employee (First_name, Last_name, Salary, Joining_date, Departament)
values ('Jerry', 'Kansxo', '6000000', '2019-01-15', 'IT'), ('Philip', 'Jose',
'8900000', '2019-02-05', 'Banking'), ...

update employee
set Salary = '1000000'
where Employee_id = '1';

insert into reward (date_reward, amount)
values ('2019-05-11', 1000), ('2019-02-15', 5000), ('2019-04-22', 2000), ('2019-
06-20', 8000);
```

Task 8: I checked both tables with SELECT *. Then I made Employee_ref_id the primary key of the reward table, with automatic numbering.

```
alter table reward
modify Employee_ref_id int auto_increment primary key;
```

5. Queries

5.1 Selecting and sorting

Task 9: I selected the first and last names of all employees.

```
select First_name, Last_name from employee;
```

Task 10: I sorted the employees by first name.

```
select * from employee order by First_name;
```

Task 11: I sorted the employees by first name (ascending) and then by salary (descending).

```
select * from employee order by First_name asc, salary desc;
```

Task 12: I selected the employees with the first name “Bob”.

```
select * from employee
where First_name = 'Bob';
```

5.2 Filters and functions

Task 13: I selected the employees with a salary larger than 3,000,000.

```
select * from employee
where salary > 3000000;
```

The query returns two employees: Philip Jose and Alex chreketo.

Task 14: I selected the employees who joined before 1 March 2019.

```
select * from employee
where Joining_date < '2019-03-01';
```

The query returns four employees.

Task 15: I counted the employees.

```
select count(*) from employee;
```

The result is 6.

Task 16: I calculated the average salary.

```
select avg(Salary) from employee;
```

The average salary is about 3,221,667.

5.3 Changing the data

Task 17: I updated the salary of the employee with the last name Kansxo.

```
update employee
set Salary = 8880000
where Last_name = 'Kansxo';
```

Task 18: I deleted all the employees from the Banking department.

```
delete from employee
where Departament = 'Banking';
```

Two employees were deleted: Philip Jose and Yohan Soso.

Task 19: I selected the two employees with the highest salaries.

```
select * from employee order by Salary desc limit 2;
```

After the changes from tasks 17 and 18, the query returns Jerry Kansxo and Alex chreketo.

6. Linking the Tables

Next, I linked the reward table to the employee table with a foreign key. The Employee_ref_id column in the reward table references the Employee_id column in the employee table.

To do this, I prepared the data first. I emptied both tables with TRUNCATE and inserted the data again. This time, I inserted 7 employees, including a new one, Bob Kinto.

```
truncate table employee;
insert into employee (First_name, Last_name, Salary, Joining_date, Departament)
values ('Bob', 'Kinto', 1000000, '2019-01-20', 'Finance'), ('Jerry', 'Kansxo',
'6000000', '2019-01-15', 'IT'), ...
```

One employee can receive more than one reward. For this reason, Employee_ref_id can’t be the primary key of the reward table. I removed the primary key and the automatic numbering from this column. Then I linked the rewards to employees 1, 2, 3 and 1, and added the foreign key.

```
alter table reward
drop primary key;

alter table reward
modify Employee_ref_id int;

alter table reward
add foreign key (Employee_ref_id) references employee (Employee_id);
```

I verified the result with SELECT * and DESC statements.

7. Join Queries

Task 20: For each reward, I calculated the difference between the reward date and the employee's joining date.

```
select r.date_reward - e.Joining_date
from employee e inner join reward r on e.Employee_id = r.Employee_ref_id;
```

I joined the two tables with INNER JOIN, using the foreign key. I used the aliases "e" and "r" to keep the query short.

Task 21: I selected the first names of the employees and the amounts of their rewards, only for rewards larger than 2000.

```
select e.First_name, r.amount
from employee e inner join reward r on e.Employee_id = r.Employee_ref_id
having amount > 2000;
```

I used HAVING to filter the result.

Task 22: I selected the first names of the employees and the amounts of all their rewards.

```
select e.First_name, r.amount
from employee e inner join reward r on e.Employee_id = r.Employee_ref_id;
```

The complete SQL script can be checked in the project repository (humanresource database.sql). The database model is available in humanresource_database_model.mwb.

8. Conclusion

In this exercise, I practised working with the structure of a database. I created and deleted databases and tables, added and removed columns, and changed primary and foreign keys. I also inserted, updated and deleted data.

The most important part was linking the two tables. I learned that a foreign key column must allow repeated values when one record (an employee) can be linked to several records (rewards). After linking the tables, I wrote join queries to combine data from both of them.